

CCCC Code Quality Report

Auto Insurance Management System

C++11 · CCCC v3.2 · 34 Modules Analysed

Report Date: 2026-04-29

34

Modules

2218

Lines of Code

658

Total MVG

65

Avg LOC/Module

19.4

Avg MVG/Module

1. Introduction

This report presents results of a static code quality analysis on the **Auto Insurance Management System** using **CCCC (C and C++ Code Counter) v3.2**. The system is a console-based C++11 application managing the full automobile insurance lifecycle — customer registration, policy issuance, claim filing, inspection, and manager approvals — with all data persisted through flat files (no DBMS).

The analysis covers **34 classes** across four architectural layers: Entities, Repositories, Services, and UI Menus. Metrics reported include LOC, MVG, WMC, CBO, DIT, and NOC per the Chidamber & Kemerer object-oriented metrics suite.

2. Project-Level Summary

Metric	Value	Threshold	Status
Number of Modules (NOM)	34	—	PASS
Total Lines of Code (LOC)	2218	—	PASS
Average LOC per Module	65.2	< 200	PASS
Total McCabe Complexity (MVG)	658	—	PASS
Average MVG per Module	19.4	< 10	NOTE
Total WMC	332	—	PASS
Average WMC per Module	9.8	< 25	PASS
Maximum WMC (Display)	23	< 25	PASS
Maximum CBO (ClaimMenu)	7	< 10	PASS
DIT across all classes	0	< 5	PASS
NOC across all classes	0	< 10	PASS

3. Per-Module Metrics

All 34 modules are listed below with measured metrics. Red values indicate the highest WMC/MVG in their layer. Rows are colour-coded by layer.

Class	Layer	LOC	MVG	WMC	CBO	DIT	NOC
Customer	Entity	28	7	13	3	0	0
Vehicle	Entity	33	10	12	3	0	0
Policy	Entity	38	11	14	3	0	0
Claim	Entity	49	13	21	3	0	0
Inspection	Entity	33	9	13	3	0	0
Staff	Entity	31	8	13	4	0	0
Workshop	Entity	29	8	11	4	0	0
User	Entity	23	6	10	4	0	0
FileHandler	Repository	11	4	0	0	0	0
CustomerRepository	Repository	73	27	10	4	0	0
VehicleRepository	Repository	60	23	9	4	0	0

Class	Layer	LOC	MVG	WMC	CBO	DIT	NOC
PolicyRepository	Repository	77	26	10	4	0	0
ClaimRepository	Repository	91	32	11	4	0	0
InspectionRepository	Repository	72	27	10	4	0	0
StaffRepository	Repository	63	23	9	4	0	0
WorkshopRepository	Repository	63	23	9	4	0	0
UserRepository	Repository	53	19	8	4	0	0
CustomerService	Service	36	10	7	6	0	0
VehicleService	Service	25	6	5	5	0	0
PolicyService	Service	41	12	8	6	0	0
ClaimService	Service	74	25	14	6	0	0
InspectionService	Service	29	7	6	5	0	0
StaffService	Service	30	8	7	6	0	0
WorkshopService	Service	29	7	6	5	0	0
AuthService	Service	46	21	9	5	0	0
CustomerMenu	UI Menu	140	44	11	4	0	0
PolicyMenu	UI Menu	107	34	7	4	0	0
ClaimMenu	UI Menu	160	59	10	7	0	0
InspectionMenu	UI Menu	79	26	6	2	0	0
StaffMenu	UI Menu	98	27	8	2	0	0
WorkshopMenu	UI Menu	74	21	7	1	0	0
ReportsMenu	UI Menu	161	54	10	6	0	0
Display	Helper	208	4	23	8	0	0
InputHelper	Helper	54	17	5	1	0	0
TOTAL	—	2218	658	332	138	0	0

Entity

Repository

Service

UI Menu

Helper

4. Analysis by Architectural Layer

Entity — 8 classes · LOC: 264 · MVG: 72 · WMC: 107

All entity classes are data containers with no inheritance (DIT=0, NOC=0). WMC ranges from 10 to 21. Claim has the highest WMC (21) due to its larger number of settable fields reflecting the claim lifecycle state machine.

Repository — 9 classes · LOC: 563 · MVG: 204 · WMC: 76

Repository classes handle all file I/O via FileHandler. CBO is uniformly 4 across all repositories. ClaimRepository is the most complex (LOC=91, MVG=32) due to additional query methods (findByStatus, findBySurveyorId). FileHandler itself has LOC=11 and WMC=0 as all methods are static.

Service — 8 classes · LOC: 310 · MVG: 96 · WMC: 62

Service classes implement business logic and sit between UI and repositories. CBO ranges from 5 to 6. ClaimService is the most complex (LOC=74, MVG=25, WMC=14) because it manages the full claim state machine. AuthService MVG=21

reflects login validation, session management, and admin seeding logic.

UI Menu — 7 classes · LOC: 819 · MVG: 265 · WMC: 59

Menu classes are the largest by LOC due to their orchestration role. ReportsMenu (LOC=161, MVG=54) and ClaimMenu (LOC=160, MVG=59) handle the most use cases. Individual WMC values stay at 6–11 — each menu method is simple but menus have many of them.

Helper — 2 classes · LOC: 262 · MVG: 21 · WMC: 28

Display (LOC=208, WMC=23) is the largest class — expected for a rendering utility covering all 8 entity types. Its MVG=4 confirms minimal branching. InputHelper (LOC=54, MVG=17) handles all safe console input with validation loops.

5. Metric Definitions

Abbr.	Full Name	Description
LOC	Lines of Code	Total executable lines excluding blank lines and comments.
MVG	McCabe Cyclomatic Complexity	Number of linearly independent paths through a method. Higher values indicate more branching.
WMC	Weighted Methods per Class	Sum of complexity of all methods in a class. Indicates overall class complexity.
CBO	Coupling Between Objects	Number of classes this class depends on. High coupling reduces maintainability.
DIT	Depth of Inheritance Tree	Levels of inheritance above this class. DIT=0 means no base class is used.
NOC	Number of Children	Number of direct subclasses. NOC=0 means no class inherits from this one.
NOM	Number of Modules	Total number of classes/modules analysed in the project.

6. Observations & Recommendations

PASS

DIT = 0 and NOC = 0 across all 34 classes.

The design is intentionally flat with no inheritance. This is appropriate for a service-oriented architecture and results in lower coupling and easier maintenance.

PASS

No class exceeds the WMC warning threshold of 25.

The highest WMC is Display (23), acceptable given it renders output for all 8 entity types. All business logic classes have WMC <= 14.

PASS

No class exceeds the CBO warning threshold of 10.

Maximum CBO is 7 (ClaimMenu). Repository classes have a uniform CBO of 4, showing clean layer boundaries. Service classes have CBO of 5-6.

PASS

Entity classes are clean and minimal.

All entity classes have WMC between 10-21 and CBO of 3-4, confirming proper separation of data from business logic.

NOTE

Average MVG per module is 19.4, above the recommended threshold of 10.

This is driven by UI Menu and Repository classes which contain many query and rendering methods. Individual methods are simple; the complexity comes from breadth, not deep branching.

NOTE Lines of Comment (COM) = 0.

No formal comment lines detected. Code relies on self-documenting naming conventions. Doxygen-style block comments are recommended for production or academic submission.

PASS ClaimService correctly centralises claim lifecycle logic.

With LOC=74, MVG=25, and WMC=14, ClaimService is the most complex service. This is expected as it manages the full 5-state claim machine: PENDING -> UNDER_INSPECTION -> APPROVED/REJECTED -> REPAIR_COMPLETE -> CLOSED.

7. Conclusion

The Auto Insurance Management System demonstrates a well-structured, maintainable codebase. All critical Chidamber & Kemerer thresholds are met: no class exceeds WMC=25 or CBO=10, and the inheritance tree is deliberately flat (DIT=0, NOC=0) consistent with a service-oriented design. The 3-layer architecture (UI -> Service -> Repository -> FileHandler -> Files) enforces clean separation of concerns and keeps inter-layer coupling low. The primary recommendation is to add Doxygen-style documentation, which would bring the project to full production-quality standards.